Formula Uno

An F1 Race Reconstruction & Analytics Engine

Complete Technical Documentation

Machine-learning lap-time modelling • XGBoost gradient boosting
Cumulative race simulation • Cross-year out-of-sample validation
Data source: FastF1 (2022–2025 seasons, ground-effect regulation era)

This document explains the system end to end: what it is, how every component works, the full build history with every issue and correction, and how to read the results.

Contents

Part 1 What This System Is

Part 2 The Data Foundation — FastF1 and the Regulation Era

Part 3 Component Deep Dive — Every File Explained

Part 4 The Build History — Step by Step, with Every Issue and Fix

Part 5 The Accuracy Improvements — Physics-Based Feature Engineering

Part 6 Results and Interpretation

Part 7 Glossary of Technical Terms

Part 1 — What This System Is

Formula Uno is a machine-learning system that **reconstructs and predicts Formula 1 race results from historical timing data**. It does not guess finishing positions directly. Instead it learns the fundamental quantity that decides a Grand Prix — **how fast each car laps under given conditions** — and then rebuilds the race lap by lap from those learned lap times, letting the finishing order emerge on its own.

The distinction matters. A model that predicts "who finishes where" learns nothing about racing; it just memorises past results. A model that predicts **lap times** learns the actual physics of the sport — how a tyre degrades, how a fuel load slows a car, how one team's package compares to another's — and can then be pointed at a season it has never seen and asked to reconstruct it. That is the core scientific claim of the project, and it is what makes the results falsifiable rather than circular.

1.1 The central question

Every design decision in this project serves one question:

> **NOTE — The research question**
>
> *If we train a model only on how cars behaved in past seasons of the current regulation era, can it accurately reconstruct a future season it has never been shown — purely from learned pace?*

This is a genuine out-of-sample test. The model trains on 2022, 2023 and 2024 and is then asked to reproduce 2025. Because 2025 had not happened when the approach was designed, there is no way for the answer to leak backwards into the model. If the reconstruction is accurate, the model has learned something real about racing. If it is not, we can see exactly which drivers and situations broke it.

1.2 The central idea: learn pace, then simulate

The system has two halves that are deliberately kept separate:

- **The learning half** — an XGBoost regression model is trained on tens of thousands of real individual laps. For each lap it sees the driver, team, circuit, year, lap number, tyre compound, tyre age, fuel load and grid position, and it learns to output the lap time in seconds.
- **The simulation half** — a race engine takes the trained model and replays a Grand Prix one lap at a time. Each driver's lap time is predicted, accumulated into a running total, and after every lap the field is re-sorted by total elapsed time to produce the running order. The finishing order is whatever the accumulated times produce at the final lap.

Keeping these halves separate is what lets us validate each one independently. We can measure how accurate the raw lap-time predictions are (lap-time RMSE) *and* how accurate the resulting race order is (position error) as two distinct numbers.

1.3 What the system deliberately is NOT

Two tempting features were considered and rejected on principle. Recording *why* is as important as the code itself.

ISSUE — Rejected: a driver/team rating system

The idea was to hand-assign skill ratings (e.g. “this car is 10/10, this driver 8/10”) and adjust results by them. It was rejected because the ratings encode the author’s personal opinion. The model would then simply reproduce those opinions back as output — a circular result that proves nothing. Data-driven conclusions must come from the data, not from a dial the author sets by hand.

ISSUE — Rejected: a hypothetical scenario simulator

The idea was to simulate “what if” races that never happened. It was rejected because such predictions are unfalsifiable — there is no ground truth to compare against, so they can never be shown right or wrong. A serious analytical project only makes claims that reality can check.

1.4 System architecture

Data flows through the system in a single direction. Each stage has one responsibility:

```
FastF1 API  -&gt;  Ingestion      (load_session.py)   raw laps + results, cached to disk
            -&gt;  Feature build  (build_features.py)  clean numeric feature table
            -&gt;  Training       (train.py)           XGBoost model, 5-fold cross-validated
            -&gt;  Simulation     (race_engine.py)     lap-by-lap race reconstruction
            -&gt;  Analytics      (compare.py)         simulated vs actual, error metrics
            -&gt;  Qualifying     (qualifying.py)      separate single-lap prediction

  The pipeline.py module wires all of the above together for one circuit
  (run_circuit) or several circuits at once (run_multi_circuit).
```

1.5 The two models

A late but important refinement was splitting the system into **two** trained models rather than one, because race laps and qualifying laps are physically different events:

	Race model	Qualifying model
Trained on	Race laps (~90s at Bahrain)	Qualifying laps (~90s flat-out)
Fuel load	Heavy, burning off each lap	Minimal (~5 kg)
Tyre use	Managed over a stint	One perfect flying lap
Purpose	Reconstruct the full race	Predict the grid order

Previously the race model was applied to qualifying laps, which produced a huge constant offset (the car is ~15–20 kg heavier assumption vs a light qualifying car). The dedicated qualifying model removed that offset entirely — see Part 5.

1.6 The validation philosophy

The project uses **cross-year temporal validation**. This is stricter than the usual random train/test split. In a random split, laps from the same race can end up in both training and test sets, letting the model “peek” at the answer. Here the split is by *time*: entire past seasons train the model, and an entire future season tests it. Nothing from the test year is ever seen during training. This mirrors how the model would actually be used in the real world — predicting a season that has not happened yet.

Part 2 — The Data Foundation

2.1 FastF1

All data comes from **FastF1**, an open-source Python library that exposes Formula 1's official timing feed: every lap, every pit stop, every tyre change, and the classified results for both races and qualifying sessions. It is the same underlying data the sport itself publishes, so the project is built on ground truth rather than scraped approximations.

Sessions are loaded with telemetry, weather and race-control messages switched off, because the model only needs lap-level information. That keeps each load fast and the cache small:

```
session = fastf1.get_session(year, event, session_type)   # e.g. 2024, "Bahrain Grand Prix", "R"
session.load(telemetry=False, weather=False, messages=False)
return {"session": session, "laps": session.laps.copy(),
        "results": session.results.copy()}
```

2.2 What a single lap record contains

Each row FastF1 returns for a lap is rich. The columns the model actually uses or filters on are:

Column	Meaning	Role in the system
Driver / Team	Three-letter code and constructor	Encoded as model inputs
LapNumber	Which lap of the race	Input; also drives fuel model
LapTime	The lap duration	The TARGET the model predicts
Compound	SOFT / MEDIUM / HARD / INTER / WET	Encoded as an ordinal input
TyreLife	Laps done on the current set	Input; drives degradation model
Position	Running position on that lap	Used to filter valid laps
PitInTime	Timestamp if the car pitted	Builds the pit-stop schedule
IsAccurate	FastF1 data-quality flag	Filters out unreliable laps
Deleted	Lap deleted (e.g. track limits)	Filters out invalid laps
TrackStatus	Green / SC / VSC codes	Filters out neutralised laps

2.3 The caching system

FastF1 caches everything it downloads to the local *data/* folder. The first time a session is requested it is fetched over the network; every subsequent request reads instantly from disk. This is why the run logs show "Using cached data" for most sessions — only genuinely new sessions (like the first pull of Bahrain 2025) hit the network. It makes iterating on the model cheap.

2.4 The regulation-era boundary: why 2022 is the earliest year

Formula 1 introduced a complete aerodynamic rules reset for the 2022 season — the "ground-effect" era. The cars generate downforce in a fundamentally different way from the 2017–2021 generation. A lap time from 2021 is produced by different physics than a lap time from 2022 onward.

NOTE — Why the training window starts at 2022

Training on pre-2022 data would teach the model a car behaviour that no longer exists, corrupting its understanding of the current cars. 2022 is therefore treated as **Year 0** of the current era. This is encoded directly as a feature (see YearEncoded in Part 3), so the model can learn how performance *develops* within the era while never mixing in obsolete physics.

Part 3 — Component Deep Dive

This part walks through every source file and explains not just what it does but *why* it does it that way.

3.1 Ingestion — load_session.py

The smallest module. It wraps FastF1 so the rest of the system never talks to the API directly. Its one clever detail is that **event** can be either a round number (an int like 1) or an event name (a string like “Bahrain Grand Prix”). That flexibility later turned out to matter a great deal — see the round-number bug in Part 4.

```
def load_race(year, event):      return _load(year, event, "R")
def load_qualifying(year, event): return _load(year, event, "Q")
```

3.2 Feature engineering — build_features.py

This is the heart of the system. It converts messy human-readable lap data into a clean numeric table an algorithm can learn from. Every feature below is a deliberate choice.

The feature list

```
FEATURE_COLS = [
    "DriverEncoded", "TeamEncoded", "TrackEncoded", "YearEncoded",
    "LapNumber", "CompoundEncoded", "TyreLife", "TyreLifeSquared",
    "FuelLoad", "GridPosition",
]
TARGET_COL = "LapTimeSeconds"
```

Every feature, explained

DriverEncoded, TeamEncoded, TrackEncoded — machine-learning models only accept numbers, not text like “VER” or “Ferrari”. A LabelEncoder maps each unique driver, team and circuit to an integer. The model then learns a per-driver and per-team pace signature from these codes. Crucially the codes carry no ordering meaning — driver 5 is not “faster” than driver 4; XGBoost treats them as categories via its tree splits.

YearEncoded — computed as *year minus 2022*. So 2022=0, 2023=1, 2024=2, 2025=3. This single number tells the model where in the regulation era a lap sits, letting it learn development trajectories: for example that a given team got faster across the years, or that lap times generally fell as the cars matured. Without it, the model would average away real year-on-year change.

LapNumber — which lap of the race it is. Carries information about track evolution (the surface “rubbers in” and speeds up over a race) and, historically, was the only proxy for fuel burn until the explicit fuel feature was added.

CompoundEncoded — tyre compound mapped to an ordinal scale: SOFT=0, MEDIUM=1, HARD=2, INTERMEDIATE=3, WET=4. The ordering is meaningful here (softer tyres are grippier but wear faster), so an ordinal encoding is appropriate rather than an arbitrary label.

TyreLife — how many laps the current set of tyres has completed. This is the single most important degradation signal: a 20-lap-old tyre is materially slower than a fresh one.

TyreLifeSquared — the square of TyreLife. Tyres do not lose grip in a straight line; degradation *accelerates* as they age and approach the "cliff." Giving the model the squared term explicitly lets it fit this curved relationship with far less data than it would need to discover the curve on its own. This was one of the accuracy improvements (Part 5).

FuelLoad — an estimate of the fuel mass on board, in kilograms, computed from lap number. A car starts a race with roughly 105 kg of fuel and burns it down steadily; roughly every kilogram costs about 0.03 seconds per lap. This was previously tangled up inside LapNumber; separating it gives the model a clean physical signal. Also a Part 5 improvement.

GridPosition — where the car started. It acts as a compact proxy for the car and driver's underlying quality on that specific weekend (a fast package tends to qualify well), helping the model anchor a driver's expected pace.

The encoders and 'training-serving symmetry'

A subtle but critical piece. The LabelEncoders are fitted **once**, on the union of every session that will ever be used — all training years plus the test year, race and qualifying together:

```
def build_encoders(sessions):
    all_drivers = pd.concat([s["laps"]["Driver"].dropna() for s in sessions]).unique()
    all_teams   = pd.concat([s["laps"]["Team"].dropna()   for s in sessions]).unique()
    all_tracks  = [s["track_name"] for s in sessions]
    return {"Driver": LabelEncoder().fit(sorted(all_drivers)),
            "Team":   LabelEncoder().fit(sorted(all_teams)),
            "Track":  LabelEncoder().fit(sorted(set(all_tracks)))}
```

> **NOTE — Why fit on the union of all sessions**
>
> If the encoder were fitted only on training years, a driver who debuts in the test year (a 2025 rookie such as Antonelli, Bearman or Hadjar) or a renamed team (Alfa Romeo becoming Kick Sauber) would be an *unseen label* and crash the encoder at prediction time. Fitting on the union guarantees every code that will ever appear already has a slot. The **same** encoder objects are then used for both training and prediction — this identical treatment is called training-serving symmetry, and breaking it is one of the most common causes of silently wrong ML results.

Filtering bad laps

Not every lap is a clean racing lap. The filter keeps only laps that are trustworthy pace samples:

- **IsAccurate** must be true — FastF1's own data-quality flag.
- **Deleted** must be false — laps struck off for track-limits violations are not representative.
- **LapTime, TyreLife, Compound, Position** must all be present.
- Compound must not be "UNKNOWN."
- **TrackStatus** must not indicate a Safety Car (code 4) or Virtual Safety Car (code 6) — those laps are artificially slow and would poison the model's idea of true pace. This was a Part 5 improvement.

The pit in-lap and out-lap are also implicitly excluded from training, because they are dominated by the pit-lane transit rather than pace. The engine handles them separately using real historical times (see 3.4).

3.3 Model training — train.py

The learner is an **XGBoost regressor** — gradient-boosted decision trees. Boosting builds many small trees in sequence, each one correcting the errors of the trees before it. It is exceptionally good at tabular

data with mixed feature types, which is exactly what we have.

```
model = XGBRegressor(
    n_estimators=2000,      # number of trees
    learning_rate=0.02,     # how much each tree contributes
    max_depth=6,            # complexity of each tree
    min_child_weight=5,     # min laps per leaf - guards against overfitting
    subsample=0.8,          # each tree sees 80% of rows (adds robustness)
    colsample_bytree=0.8,   # each tree sees 80% of features
    random_state=42, n_jobs=-1)
```

Cross-validation. Before fitting the final model, training runs 5-fold cross-validation: the data is split into five parts, the model trains on four and is tested on the fifth, rotating through all five. The averaged error ("CV RMSE") is an honest estimate of how the model performs on data it did not train on, computed *before* any race simulation. A low training error with a much higher CV error would be the classic warning sign of overfitting.

> **NOTE — The learning-rate / tree-count trade-off**
>
> The hyperparameters were tuned from a fast default (500 trees at 0.05 learning rate) to a slower, more thorough setting (2000 trees at 0.02). A smaller learning rate with more trees generally reaches a better optimum — it takes smaller, more careful steps. The **min_child_weight=5** addition forces every decision-tree leaf to be backed by at least five real laps, which stops the model from carving out overly specific rules that fit noise.

3.4 The race engine — race_engine.py

This module turns a lap-time model into a race. It is the most intricate part of the system.

DriverState

Each car is represented by a mutable **DriverState** that evolves through the race — it carries the current compound, tyre age, cumulative time, running position, the list of lap times so far, and whether the car has retired. The simulation is essentially the evolution of twenty of these objects across the race distance.

The pit-stop schedule

Before the race starts, the engine scans the real lap data for pit stops and builds a schedule: for each stop it records the in-lap, the out-lap, the new compound fitted, and the **actual historical times** for those two laps.

> **NOTE — Why pit laps use real times, not the model**
>
> The model was never trained on in-laps and out-laps — they were filtered out because they are dominated by pit-lane transit, not pace. Asking the model to predict them would be asking it about something it never learned. Using the real historical times instead cleanly separates *strategy accuracy* from *speed-model accuracy*, so each can be judged on its own.

The lap loop

For every lap, for every active driver, the engine decides which of three cases applies:

- **Out-lap** (car rejoining after a stop): use the real historical out-lap time.
- **In-lap** (car pitting at the end of this lap): use the real historical in-lap time, then switch the car to its new compound and reset tyre age to zero.

- **Normal lap**: ask the model to predict the lap time from the driver's current state.

The normal-lap prediction assembles exactly the same ten features used in training, in the same order, including the freshly computed fuel load and squared tyre life:

```
fuel_load    = 105.0 - (lap_number - 1) * (105.0 / total_laps)
tyre_life_sq = state.tyre_age ** 2
features = {"DriverEncoded": ..., "TeamEncoded": ..., "TrackEncoded": ...,
            "YearEncoded": ..., "LapNumber": ..., "CompoundEncoded": ...,
            "TyreLife": state.tyre_age, "TyreLifeSquared": tyre_life_sq,
            "FuelLoad": fuel_load, "GridPosition": ...}
```

From lap times to positions

After every driver has completed a lap, the engine adds that lap time to each car's cumulative total and then **sorts the field by cumulative time** — the car with the lowest total elapsed time is leading. This is the key modelling choice: *positions are never set directly, they emerge from accumulated time*, exactly as in a real race. A driver who laps two tenths quicker every lap will slowly climb the order on their own.

> **ISSUE — The known limitation: the 'ghost car'**
>
> The current engine keeps every car circulating for the full distance. A driver who retired or was disqualified in reality still accumulates lap times in the simulation — a 'ghost car.' This is a deliberate, understood simplification: the model reconstructs *pace*, and mechanical failures or administrative penalties are outside its scope. It is why a retired car can appear in the simulated order where reality classified it last.

3.5 Analytics — compare.py

This module scores the simulation against reality with two independent metrics:

- **Position comparison** — simulated finishing position versus actual, per driver, plus summary counts (exact positions, within-two positions, mean position error).
- **Per-driver lap-time RMSE** — root-mean-square error between each simulated lap and the real lap, computed only over clean (IsAccurate) laps so the comparison is apples-to-apples with training. The overall 'Fleet RMSE' aggregates this across all drivers.

Reporting both is important: a simulation can get lap times almost perfect yet shuffle a few positions (because midfield cars are separated by tiny margins), or get positions right while individual lap times drift. Two numbers tell a truer story than one.

3.6 Qualifying — qualifying.py and the dedicated model

Qualifying is validated separately. First the fastest clean lap for each driver is extracted from the qualifying session — that single lap is what set their real grid slot. Then the **dedicated qualifying model** predicts a time for each of those laps, and drivers are ranked by predicted time to produce a predicted grid, which is compared against the real grid.

Because the qualifying model is trained on real qualifying laps (light fuel, one flying lap), its predicted times are on the correct absolute scale — unlike the earlier approach of borrowing the race model, which produced times ~15–20 seconds too slow. The ranking was always reasonable; the dedicated model fixed the absolute accuracy too.

3.7 The orchestrator — pipeline.py

run_circuit ties everything together for one track: it loads every training-year race and qualifying session plus the test-year sessions, builds shared encoders across all of them, builds the race and qualifying feature tables, trains both models, simulates every training year as an in-sample sanity check, simulates the test year as the real out-of-sample test, and finally runs the qualifying prediction. **run_multi_circuit** does the same but trains a single combined model across several circuits at once, so knowledge from one track can inform another.

Part 4 — The Build History

This part is the chronological story of how the system was built, including every issue that surfaced and the correction made for it. The bugs are as instructive as the features — each one exposed something real about the data or the method.

Phase 1 — Foundations: ingestion and first features

The first work was loading a single race with FastF1 and inspecting exactly what one lap record contains, then writing the feature pipeline to turn those records into a numeric table. The initial feature set was seven columns (driver, team, track, lap number, compound, tyre life, grid position) predicting lap time in seconds.

ISSUE — Unary-not on a None value

The bad-lap filter tried to invert the **Deleted** column with **~laps["Deleted"]**. FastF1 stores *None* rather than *False* for laps that were never deleted, and Python cannot apply a bitwise-not to None — it raised *TypeError: bad operand type for unary ~: NoneType*.

FIX — Fill the blanks before inverting

Changed to **~laps["Deleted"].fillna(False).astype(bool)**, which converts the missing values to False first, so the inversion is always operating on real booleans.

Phase 2 — The race engine and the simulation model

Next came the engine that replays a race from predicted lap times, accumulating time and deriving positions. This is where the decision was cemented that the system simulates *lap times* and lets finishing order emerge, rather than predicting order directly. Pit stops were wired in using real historical in/out-lap times.

ISSUE — The duplicate-column feature-count mismatch

LapNumber was accidentally listed both in the feature columns and in the display columns kept for reporting. Pandas returned the column twice, so the feature matrix had eight columns where the model expected seven. The model then refused to predict, complaining of a shape mismatch.

FIX — One home for each column

Removed LapNumber from the display list, since it already lived in the feature list. A stale saved model from the broken run also had to be deleted and retrained, because it had baked in the wrong feature count.

ISSUE — Windows terminal could not print the arrow character

Progress messages used a Unicode arrow. The Windows console's default code page could not encode it and every run crashed with a *UnicodeEncodeError*.

FIX — Use plain ASCII in console output

Replaced the arrow glyph with a simple **->** throughout the print statements.

Phase 3 — Cross-year validation and the multi-year model

The single-year approach exposed a conceptual problem: a model trained on one season cannot know about the next season's car development. The fix was to train across **2022, 2023 and 2024 together** and add the YearEncoded feature so the model could learn how performance develops within the regulation era, then test on 2025.

ISSUE — New drivers and renamed teams crashed the encoder

Extending to multiple years surfaced drivers and team names that did not exist in the earliest training year. A LabelEncoder fitted on only the training years treated these as unseen labels and raised a ValueError during prediction.

FIX — Fit encoders on the union of all sessions

build_encoders was changed to fit on every session at once — all training years plus the test year — so every driver, team and track that will ever appear already has an integer code.

ISSUE — YearEncoded missing from qualifying prediction

When YearEncoded was added to the model, the qualifying prediction path still built its feature vector without it, so the columns no longer matched the trained model and it raised *KeyError: YearEncoded not in index*.

FIX — Thread the year through every prediction path

Added year and reg_era_start parameters to predict_qualifying and included YearEncoded in its feature dictionary. The corresponding calls in pipeline.py were then updated to pass the test year through — in both run_circuit and run_multi_circuit.

Phase 4 — Domain corrections from real F1 knowledge

Some 'errors' were not model failures at all but the model correctly capturing pace that reality then overruled administratively.

NOTE — The 2024 Belgian Grand Prix disqualification

The model simulated George Russell as a frontrunner at Spa 2024, whereas the official results classified him last. Russell had in fact **won the race on the road** but was disqualified afterwards because his car was found to be a kilogram under the minimum weight; Lewis Hamilton inherited the win. The model's pace prediction was correct — the discrepancy was the 'ghost car' limitation meeting a post-race penalty, not a speed error.

Phase 5 — The accuracy overhaul

The most recent phase added physics-based features (fuel load, squared tyre life), filtered out Safety-Car and Virtual-Safety-Car laps, tuned the hyperparameters, and split off a dedicated qualifying model. These are covered in full in Part 5.

ISSUE — The wrong circuit — round 1 is not always Bahrain

The Bahrain runner used **event=1**, the round number. In 2022–2024 round 1 was indeed Bahrain, but the **2025 season opened in Australia**. So every '2025 Bahrain' result was silently computed on Australian Grand Prix data. The giveaway was the qualifying times: the reported figures were around 75 seconds, but a real Bahrain qualifying lap is around 88–90 seconds — the Albert Park lap in Melbourne is simply shorter.

FIX — Address races by name, not by round number

Switched the runner to **event="Bahrain Grand Prix"**, which FastF1 resolves to the correct race in every season regardless of where it falls on the calendar. The stale models built on the wrong data were deleted and everything was retrained. The 2025 out-of-sample lap-time RMSE immediately dropped from an apparent ~5.9 s to a genuine ~1.5 s, because the model was finally being tested on the circuit it was trained on.

Part 5 — The Accuracy Improvements

Five changes were made to push predicted lap times closer to reality. Each is grounded in the actual physics of an F1 car, which is why they help the model rather than just complicate it.

5.1 Fuel load — the biggest single win

A Formula 1 car starts a race carrying roughly 105 kilograms of fuel and burns it off steadily over the distance. Extra mass slows the car everywhere — corners, braking, acceleration — by roughly 0.03 seconds per lap per kilogram. Over a full race the car is around three seconds a lap faster at the end than at the start, from fuel burn alone.

Previously the model had to infer this from LapNumber, where it was hopelessly tangled with tyre degradation (later laps mean both a lighter car *and* older tyres, pulling lap time in opposite directions). Splitting fuel out as its own feature gives the model two clean, separable signals:

```
FuelLoad = 105.0 - (LapNumber - 1) * (105.0 / total_laps)
```

The exact same formula is used in the race engine at prediction time, so training and simulation stay perfectly consistent.

5.2 Non-linear tyre degradation

Tyres do not lose grip at a constant rate. A stint has three phases: a brief warm-up, a long stable plateau, and then an accelerating drop-off as the rubber passes its useful life — the ‘cliff.’ A purely linear TyreLife feature cannot represent that curve. Adding **TyreLifeSquared** gives the model the curvature it needs to model accelerating wear, which it can now fit with far less data.

5.3 Filtering Safety-Car and Virtual-Safety-Car laps

Under a Safety Car or VSC the whole field is forced to slow dramatically — a lap can be 10–20 seconds slower than normal. If even a handful of these survive into training, they teach the model that a particular lap-and-tyre combination is far slower than it truly is on a green track. The filter now explicitly drops laps whose TrackStatus signals a Safety Car (4) or VSC (6), cleaning a source of outlier corruption.

5.4 Hyperparameter tuning

The XGBoost settings were moved from a quick default to a more thorough configuration: the number of trees was raised from 500 to 2000 while the learning rate was lowered from 0.05 to 0.02, and min_child_weight was set to 5. Smaller, more numerous steps reach a better solution, and the minimum-leaf constraint reduces overfitting to individual noisy laps.

5.5 A dedicated qualifying model

The single most visible fix for qualifying. Instead of applying the race model to qualifying laps, a second model is now trained specifically on historical qualifying laps, with fuel fixed at a light ~5 kg and grid quality taken from the qualifying result. Because it learns the real qualifying regime, its predicted times land on the correct absolute scale.

FIX — The measured effect at Bahrain

Qualifying lap-time error fell from roughly 21 seconds (race model borrowed for qualifying, and on the wrong circuit) to about **1.0 second** with the dedicated model on the correct circuit. The predicted pole time of 89.95 s sat just 0.11 s off Piastri’s actual 89.84 s.

Part 6 — Results and Interpretation

6.1 In-sample accuracy improves year over year

Simulating the training years back is a sanity check — the model should reproduce races it learned from. At Bahrain the reconstruction gets sharper as more era data accumulates:

Year (in-sample)	Mean position error	Fleet lap-time RMSE	Exact positions
2022	4.60 places	0.272 s	2 / 20
2023	2.80 places	0.247 s	2 / 20
2024	0.50 places	0.162 s	12 / 20

The 2024 reconstruction is near-perfect — half a position of average error and lap times correct to about a sixth of a second. This confirms the model genuinely learned the era's pecking order.

6.2 The real test: 2025 out-of-sample

On the season it had never seen, at the correctly identified Bahrain circuit, the model produced a mean position error of **2.90 places** and a fleet lap-time RMSE of **1.49 seconds** — predicting lap times to within about one and a half seconds on a completely unseen year. Several positions were reconstructed exactly (Hamilton P5, Gasly P7, Lawson P16 among them).

6.3 What the remaining error is made of

The errors are not random — they cluster on precisely the situations the model could not have learned:

- **Drivers who changed teams** — Carlos Sainz moving to Williams is still read partly through his Ferrari-era pace, because the model has three years of him in a Ferrari and none in a Williams.
- **Rookies with no history** — Antonelli, Bearman, Bortoleto, Doohan, Hadjar and Lawson have little or no prior race data in the training window, so the model has nothing to learn their pace from and falls back on nearby signals such as grid position.

6.4 The one problem no feature can fix

> **NOTE — Why the rookie error is fundamental**
>
> You cannot predict a driver's pace from data that does not exist. For a driver who never raced in 2022–2024, the model has zero laps to learn from, so it generalises from the nearest driver it does know — which is inevitably imperfect. No amount of feature engineering creates information that was never recorded. The real-world remedies are external: pre-season testing data, or junior-formula (F2) results as a prior. This is a data-availability ceiling, not a model flaw.

6.5 Qualifying

With the dedicated model on the correct circuit, the 2025 Bahrain qualifying prediction reached a lap-time RMSE of about 1.0 second and placed 15 of 20 drivers within two grid positions of their real slot, with a mean position error of 1.9 places — a strong result for single-lap pace, which is far noisier than race pace.

Part 7 — Glossary of Technical Terms

XGBoost	A gradient-boosting library that builds an ensemble of decision trees in sequence, each correcting the previous ones' errors. Strong on tabular data.
Regression	Predicting a continuous number (here, lap time in seconds) rather than a category.
RMSE	Root-mean-square error. The typical size of the model's prediction error, in the same units as the target (seconds). Lower is better; it penalises large misses heavily.
Cross-validation	Splitting data into folds and rotating which fold is held out for testing, to estimate performance on unseen data honestly.
Out-of-sample	Data the model was never trained on. The 2025 season is the out-of-sample test here.
LabelEncoder	A tool that maps text categories (driver, team, track) to integers so a model can use them.
Training-serving symmetry	Using the exact same data transformations at prediction time as at training time. Breaking it silently corrupts results.
Feature	A single input signal the model reads — e.g. tyre life or fuel load.
Target	The value the model is trained to predict — here, lap time in seconds.
Ground effect	The 2022 aerodynamic regulation that generates downforce under the car's floor; the defining trait of the current era and the reason training starts in 2022.
Degradation	The gradual loss of tyre grip (and therefore lap time) as a set of tyres ages.
In-lap / out-lap	The lap entering the pits and the lap leaving them; dominated by pit transit, so handled with real times rather than the model.
Fleet RMSE	The lap-time RMSE aggregated across all drivers in a race.
Ghost car	A retired or disqualified car that the simulation keeps running for the full distance, because the model reconstructs pace only.

Formula Uno • a data-driven reconstruction of Formula 1, validated against reality it was never shown.